

UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

Lesson 6: k-NN, Naive Bayes and Support Vector Machines

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

13 November 2026

Before we start

- Exercise 4: which threshold did you choose for the classifier, and how did you defend it?
- Exercise 5: the score that looked too good. Which problems did you find behind it, and what score was left once you had fixed them?

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Today: the first lesson that offers a choice

- **k-nearest neighbours (k-NN)**, and the **curse of dimensionality** that limits it
- **Naive Bayes**: a probability model, very cheap to train
- **Support vector machines (SVM)**: the widest margin, then kernels
- How to choose a model family, and why that matters more than tuning

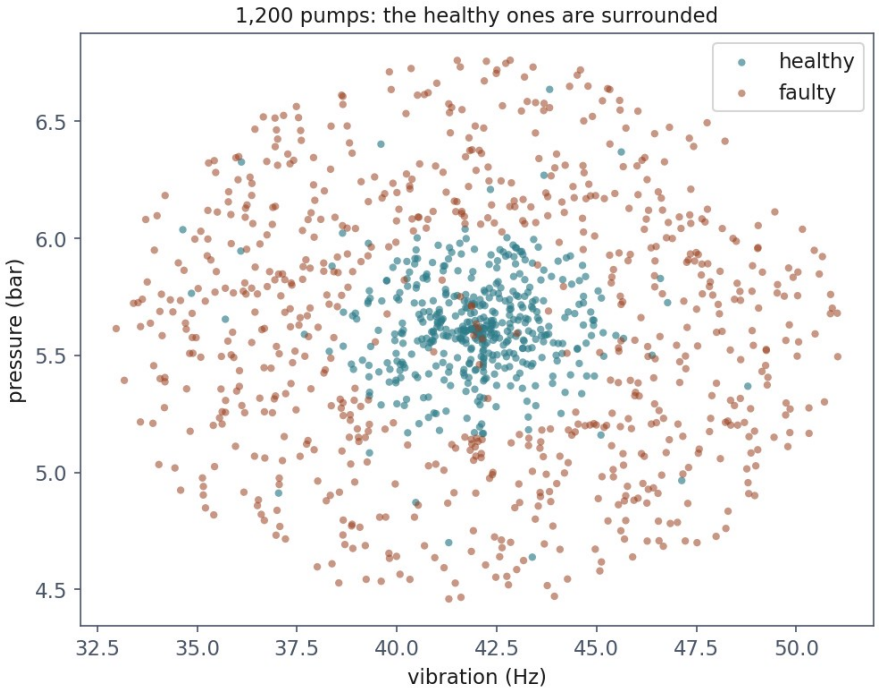
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

1,200 pumps, two readings each

- 1,200 simulated pumps, two sensor readings each: vibration in hertz (Hz), pressure in bar
- A pump is faulty when its readings fall **outside the design envelope** (the range it was built to run in), too low as readily as too high
- 735 of the 1,200 are faulty: **61.3%**
- 4% of the labels are flipped on purpose, as sensor errors: no model can score above about **0.96** (the noise ceiling)

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The shape of the problem: healthy pumps in the middle, faulty all around



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What a straight boundary costs

Model	Accuracy, 5-fold cross-validation (mean ± sd over the folds)
Always predict the majority class: 735 ÷ 1,200	0.613
Logistic regression (lesson 4)	**0.613 ± 0.000**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

k-nearest neighbours: the whole algorithm

- A line fails, so try a method that assumes no shape: to classify a new pump, take the majority vote of the **k closest** training pumps
- There is no training step: “fitting” only **stores the data** (a **lazy learner**: the work waits until a prediction is asked for)
- In code:

```
KNeighborsClassifier(n_neighbors=5)
```


UNIVERSITÀ DEGLI STUDI DELL'AQUILA

“Closest” means the smallest Euclidean distance

$$d(x, x') = \sqrt{\sum_{j=1}^{n} (x_j - x'_j)^2}$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Scale first, or vibration decides everything

- Vibration varies over about **18 Hz**, pressure over about **2.3 bar**
- Unscaled, pumps 10 Hz apart are at distance 10, pumps 1 bar apart at distance 1: vibration decides who the neighbours are
- The fix (lesson 2): **standardise** each column: subtract its mean, divide by its standard deviation
- In code: `make_pipeline(StandardScaler(), KNeighborsClassifier(5))`

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

k sets the balance between bias and variance

- **Small k**: the vote follows each training point, mislabelled ones included. Low bias, high variance (lesson 5)
- **Large k**: a vote over a wide neighbourhood smooths away real structure too. High bias, low variance
- At k = 1 the training accuracy is **exactly 1.000**: each training pump is its own nearest neighbour

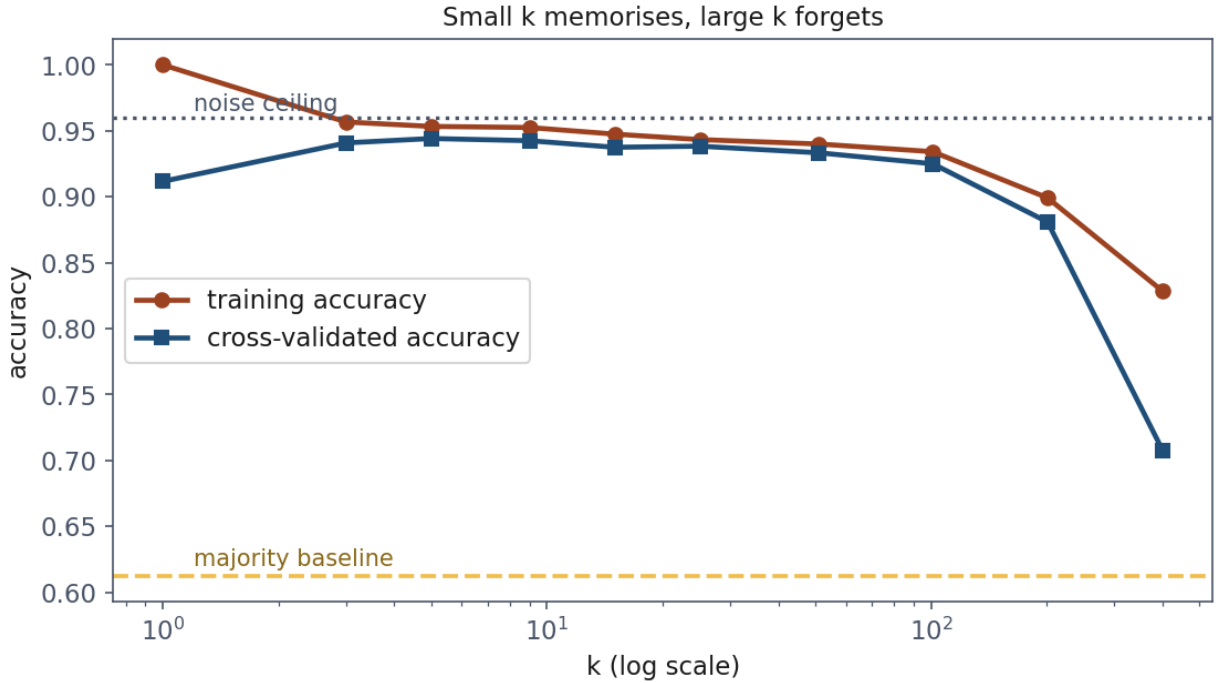
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Measured on the pumps: training against cross-validated accuracy

k	Training accuracy	Cross-validated accuracy, 5 folds
1	**1.000**	0.912
5	0.953	**0.944**
15	0.948	0.938
51	0.940	0.933
401	0.828	0.708

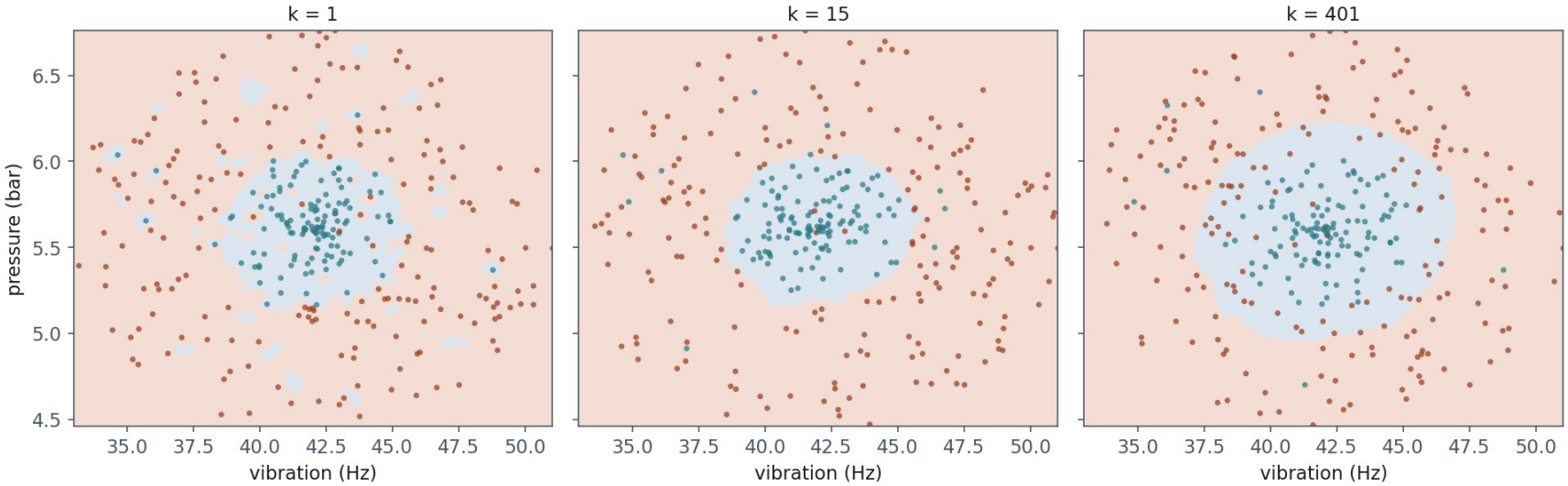
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Accuracy against k, between the baseline and the noise ceiling



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The boundary k-NN draws at k = 1, 15 and 401



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What k-NN costs

- Training costs nothing; the cost comes at **every prediction** instead
- A direct implementation compares each new pump with **every** training row: about m × n operations for m rows and n features
- To predict, the model needs the whole training set: **deploying the model means handing over the data**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

An experiment: add columns of pure noise

- Keep the 1,200 pumps and their two real readings **unchanged**
- Add columns of **pure noise**: normal random numbers, unrelated to the label
- Only the number of columns changes: the problem is as solvable as before
- Same model each time: standardised k-NN, k = 15, 5-fold cross-validation

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The two real readings are still there, and accuracy still falls

Columns: 2 real + noise	Cross-validated accuracy	Accuracy − baseline 0.613
2	0.938	+0.325
12	0.762	+0.149
27	0.662	+0.049
52	0.602	**−0.011**
102	0.578	−0.035

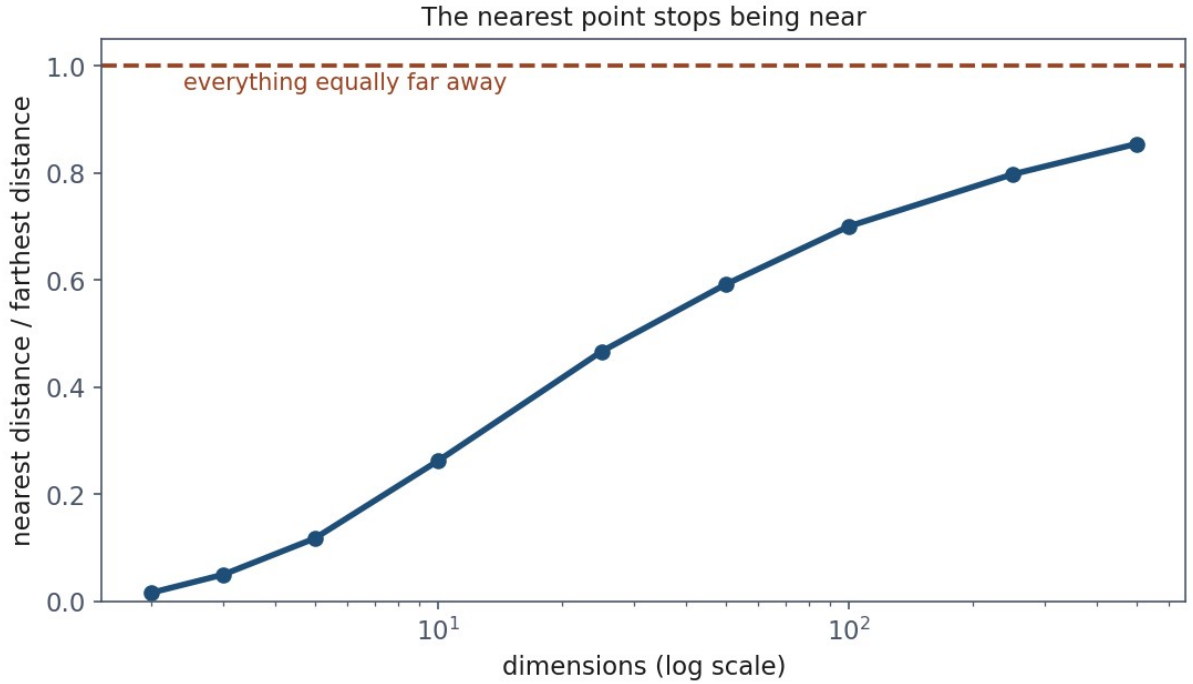
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Why: distances stop varying

- Scatter 1,000 random points in a cube; for a new point, measure the distance to its **nearest** and its **farthest** neighbour
- In two dimensions the two differ a lot: that is what makes “nearest” mean something
- Each extra dimension adds a squared difference to every distance: over many dimensions **all distances come out about the same**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The nearest point stops being near



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Nearest divided by farthest

Dimensions	nearest distance ÷ farthest distance, mean over 60 points
2	0.016
10	0.263
50	0.592
100	**0.701**
500	0.855

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

In 100 dimensions the nearest point is 70% as far as the farthest

- In **two** dimensions the ratio is 0.016: the nearest point is at **2%** of the farthest distance, so “nearest” picks out something special
- In **one hundred** it is 0.701, **70%**: the nearest point is barely nearer than a point picked at random
- So a vote among “the five nearest” is almost a vote among five pumps taken at random: that is why 52 columns fell to the baseline

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What the curse is, and is not

- **Not** that problems with many columns cannot be learned: a linear model gives a useless column a coefficient near zero, and loses almost nothing
- It is that **methods built on distances stop working**, because every distance comes out about the same
- For k-NN, every column is part of the distance: choose the columns before any distance is computed, inside the pipeline

Notebook 1, live

section, and its slides	you call (→ the number to come back with)
the straight line, 4-6	`LogisticRegression` scored by `cross_val_score` over `StratifiedKFold` folds → 0.613
k-NN, scaled first, 7-9	`make_pipeline(StandardScaler(), KNeighborsClassifier(k))`
choosing k, 10-13	`cross_val_score` for each k → 0.944 at k = 5
columns of noise, 15-16	the pipeline with k = 15 on `load_with_noise(d)` → 0.602 at 52 columns
distances stop varying, 17-20	NumPy only: nearest ÷ farthest → 0.701 in 100 dimensions

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Break

- Twelve minutes

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Naive Bayes: turn the question around

- A second family, built on probabilities: we want P(y = c | x), the probability of class c **given** the readings x
- Bayes' rule rewrites it with what the data can give: how common each class is, and how its readings are spread
- The hard part: how likely **this exact combination of readings** is among pumps of that class

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Bayes' rule, applied to a class

$$P(y = c \mid x) = \frac{P(x \mid y = c)\ P(y = c)}{P(x)}$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Drop the denominator: it is the same for every class

$$\hat{y} = \arg\max_c P(x \mid y = c)\, P(y = c)$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The assumption: independent, given the class

- To make P(x | y = c) estimable, Naive Bayes assumes: **given the class, the features are independent of one another**
- Then the density of all n readings together is the product of n one-dimensional densities, one per feature
- Each one-dimensional density is estimated from the pumps of that class alone. In code:

```
GaussianNB()
```


UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The assumption, written: a product of n densities

$$P(x \mid y = c) = \prod_{j=1}^{n} P(x_j \mid y = c)$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Gaussian: each factor is a bell curve

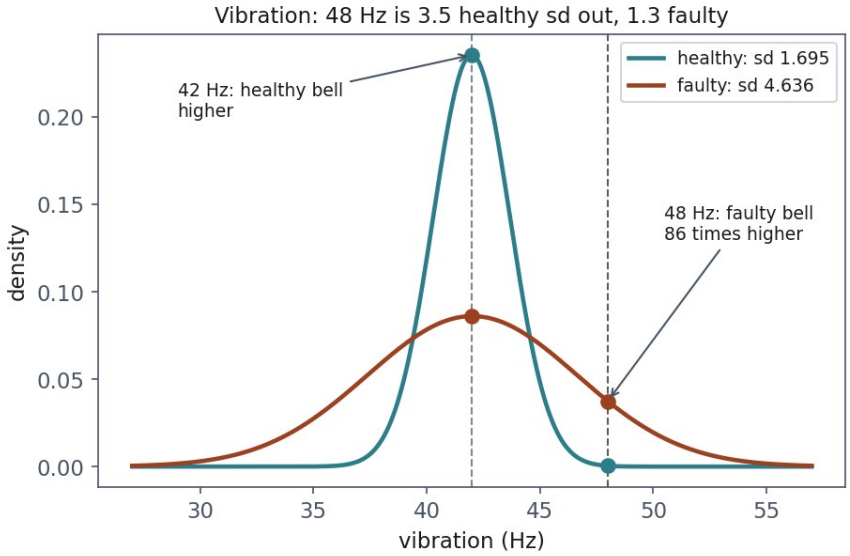
$$P(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left[-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right]$$

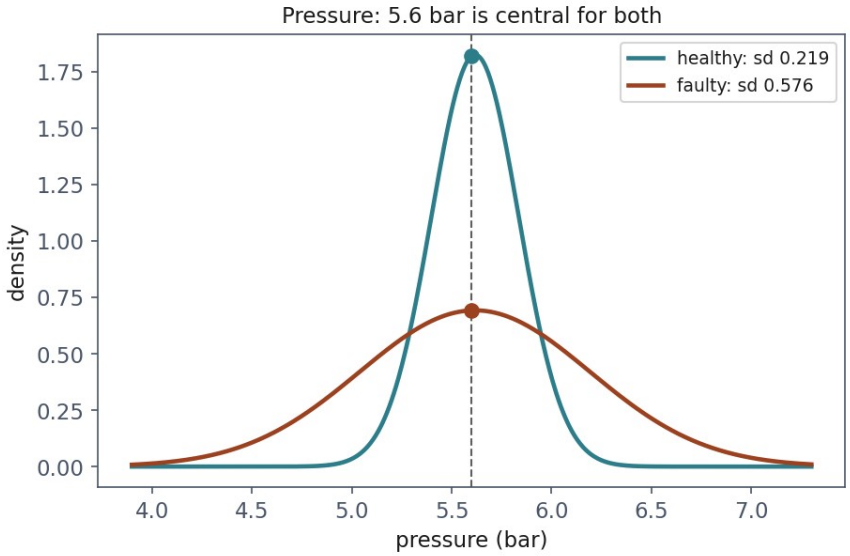
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Training computes ten numbers: the widths separate the classes

Fitted on the 1,200 pumps	Healthy	Faulty
Prior: the class's share of the pumps	465 ÷ 1,200 = 0.3875	735 ÷ 1,200 = 0.6125
Vibration mean	41.986 Hz	42.015 Hz
Vibration standard deviation	**1.695 Hz**	**4.636 Hz**
Pressure mean	5.617 bar	5.616 bar
Pressure standard deviation	**0.219 bar**	**0.576 bar**

A pump at 48 Hz, 5.6 bar: there the faulty bell is 86 times higher





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Take logs: the product becomes a sum

$$\ln\left[P(y=c)\prod_j P(x_j \mid y=c)\right]$$

$$= \ln P(y=c) + \sum_j \ln P(x_j \mid y=c)$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

48 Hz, 5.6 bar: scores and their logs

Class	Row	Prior	Vibration density	Pressure density	Product of the three, or sum of their logs
Healthy	value	0.3875	0.000433	1.818	0.000305
Healthy	ln	−0.948	−7.745	+0.598	−8.095
Faulty	value	0.6125	0.0374	0.692	0.01585
Faulty	ln	−0.490	−3.286	−0.368	**−4.144**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

From the two totals to probabilities

Step	Healthy	Faulty
1. Log total, previous slide	−8.095	−4.144
2. Undo the log: e^total = prior × densities	0.000305	0.01585
3. Divide each by their sum, 0.000305 + 0.01585 = 0.01616 = P(x)	**0.019**	**0.981**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Software uses only the gap between the two log totals

- With many features, e^total rounds to 0 for both classes, and 0 ÷ 0 cannot be computed
- The gap between the log totals stays an ordinary number: $-4.144 - (-8.095) = 3.951$
- P(faulty) = 1 / (1 + e^(−3.951)) = 1 / 1.0192 = **0.981**, as before
- e^3.951 ≈ 52: faulty is 52 times as likely as healthy (**odds of 52 to 1**)

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

A cheap model, built on an assumption that is almost never true

- The gain: training is **one pass** over the data; extra features cost almost nothing
- The assumption rarely holds: vibration and pressure both depend on how hard the pump runs
- In a text, “New” and “York” appear together far more than independence allows
- So the useful question: **when does being wrong about it cost nothing?**

Naive Bayes on the pumps: 0.933

Model	Accuracy, 5-fold cross-validation, same folds
Majority baseline	0.613
Logistic regression	0.613
Gaussian Naive Bayes	**0.933**
k-NN, k = 5	0.944

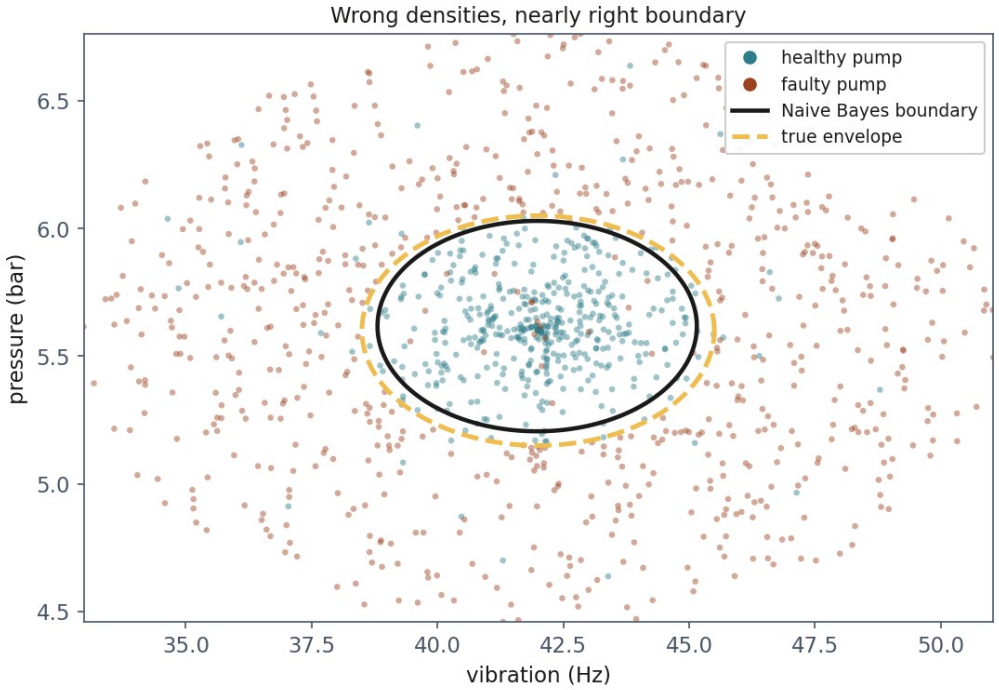
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Uncorrelated within each class, but not independent

Correlation, within one class	between vibration and pressure	between the two standardised readings, squared (how far from the centre)
healthy pumps	−0.006	+0.168
faulty pumps	−0.049	**−0.420**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The densities are wrong, but the boundary they draw is about right



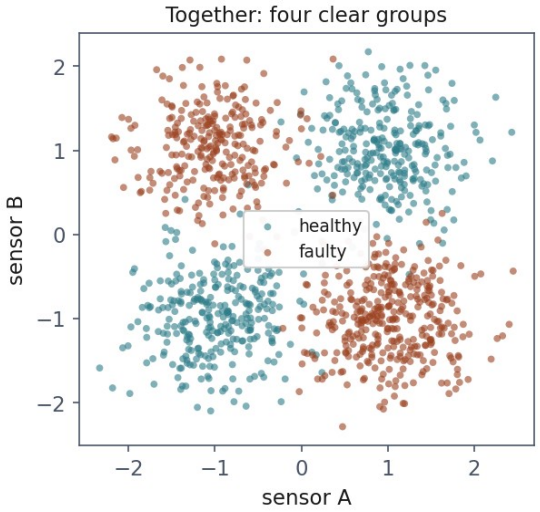
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

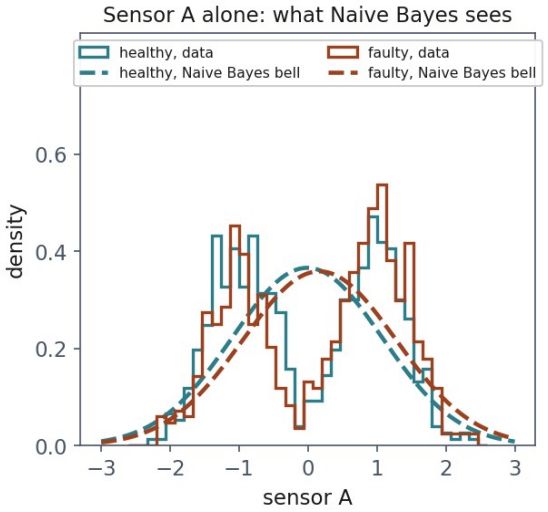
A second dataset: the signal is an interaction

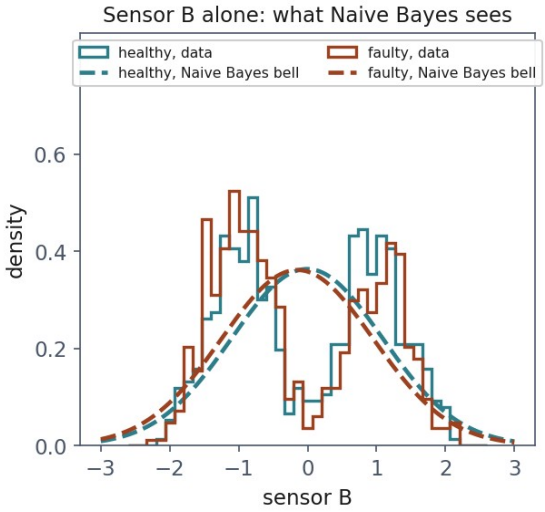
- 1,200 more simulated pumps, with two other sensors, A and B
- Faulty when **exactly one** of the two readings is high
- Both high: high-load mode. Both low: idle. One alone: demand and delivery do not match
- So A alone says nothing: what a high A means depends on B (an **interaction**)

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Each sensor alone: the two classes look the same







UNIVERSITÀ DEGLI STUDI DELL'AQUILA

0.404: below the baseline, and below chance

Model	Accuracy, 5-fold cross-validation
Majority baseline: 628 faulty ÷ 1,200	0.523
Gaussian Naive Bayes	**0.404**
Logistic regression	0.393
SVM, linear kernel	0.606
k-NN, k = 5	0.967
SVM, radial basis function (RBF) kernel	0.972

Its probabilities cannot be trusted

- Interacting sensors, 300 test pumps. Confidence (the probability given to the predicted class): **0.567** when right, **0.555** when wrong
- So it cannot tell when it is wrong
- Features that copy each other push it the other way: towards 0 or 1
- It may still rank pumps usefully (lesson 4), but its **probabilities are not calibrated**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Notebook 2, live

section, and its slides	you call (→ the number to come back with)
on the pumps, 37	`GaussianNB` beside `KNeighborsClassifier` and `LogisticRegression` → 0.933
what training computed, 29-35	`.theta_`, `.var_`, `.class_prior_`, `predict_proba` → 0.981 at 48 Hz
checking the assumption, 38-39	`X[y == c].corr()`, within each class → −0.420
the interacting sensors, 40-42	the same models, plus `SVC` → 0.404
confidence, right and wrong, 43	`train_test_split`, then `predict_proba` → 0.567 against 0.555

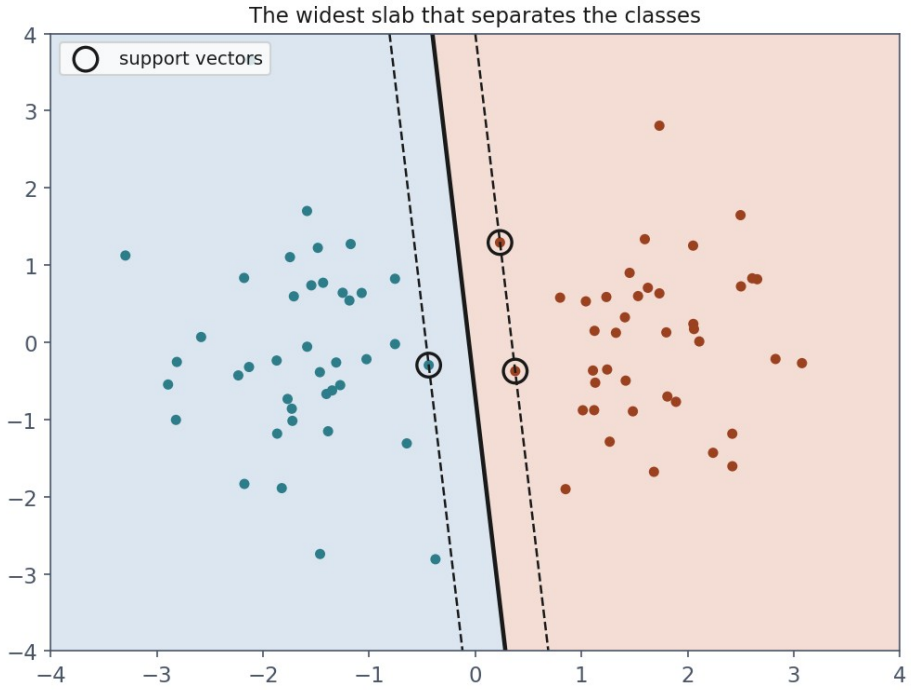
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The margin: choosing among lines that all fit

- Third family. When a line can separate two classes, **infinitely many** lines do, all with **zero training error**: the error cannot choose
- The SVM picks the line with the widest **margin**: an empty strip (the slab) around the line, widened until it touches a point of each class
- The reason: a line that passes close to a point gets it wrong after a small change in that point's readings

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

80 points in two groups: three of them fix the line (support vectors)



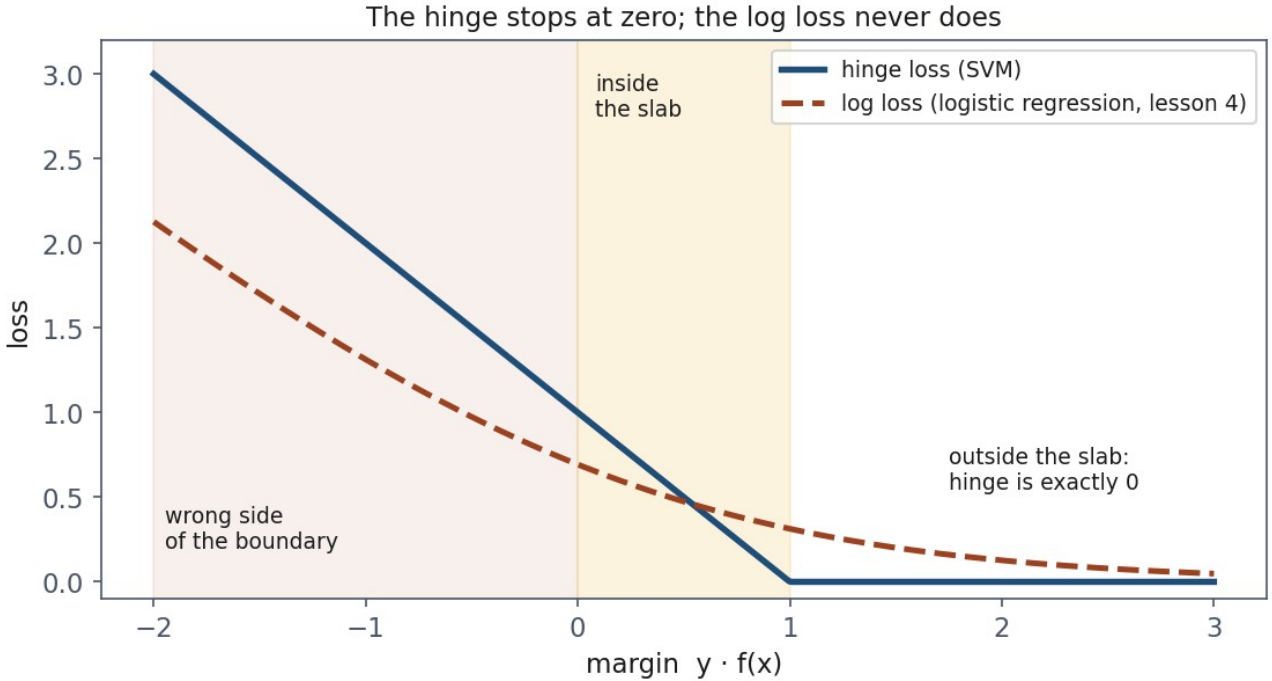
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Not separable? Charge for the violations

- With 4% of the labels flipped, no slab separates the pumps: the strict problem has **no solution**
- So let point i enter the slab by a slack ξ_i, and charge C per unit
- $\xi_i = 0$ outside the slab, 0 to 1 inside it, above 1 on the wrong side
- That charge is the **hinge loss** $\max(0, 1 - y \cdot f(x))$, labels ± 1, $f(x) = w \cdot x + b$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The hinge loss stops at zero; lesson 4’s log loss never does



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The SVM objective: a wide slab (small w), plus C times the slack

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i$$

subject to $y_i(w^\top x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

C is the price of a training error

- **Large C**: slack is expensive, so the slab narrows to classify more training points. Low bias, high variance
- **Small C**: slack is cheap, so the slab stays wide and some training points stay wrong. Higher bias, lower variance
- C plays the role of k in k-NN and of λ in lesson 3
- But **large C means less regularisation**: the opposite of λ

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The best straight line is still a straight line

Model, on the pumps	Accuracy, 5-fold cross-validation	Support vectors, fitted on all 1,200
SVM, linear kernel: a straight line	**0.613 ± 0.000**	947 of 1,200 (**79%**)
SVM, RBF kernel: a curved boundary	0.947 ± 0.005	278 of 1,200 (23%)

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Many support vectors: a free warning

- The linear SVM, `SVC(kernel="linear")`, keeps **79%** of the pumps as support vectors; the RBF SVM keeps **23%**
- 79% means almost every pump sits on or inside the slab: no strip of empty space separates the classes
- The count comes free after `fit`, as `len(model.support_)`: a high share is a warning before any cross-validation

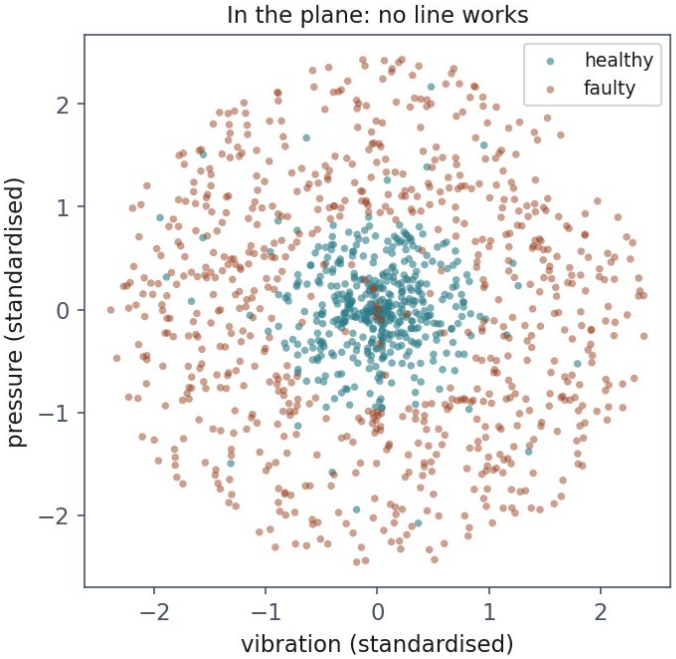
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

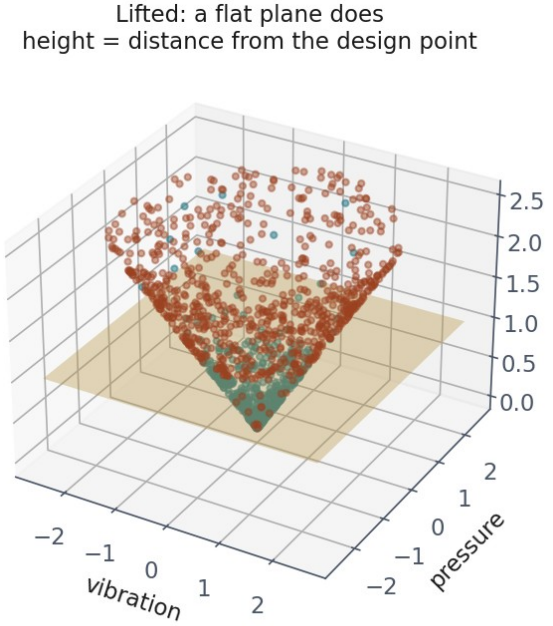
A new coordinate makes the classes separable

- In the plane, healthy pumps sit in a disc and faulty ones around it: no line separates them
- Add a third coordinate, computed from the two readings: each pump’s **distance from the design point**
- Healthy pumps rise a little, faulty ones a lot, and a flat plane separates them
- The classes were separable all along, in other coordinates

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The same pumps, lifted





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The SVM’s solution: w is a weighted sum of the training pumps

$$w = \sum_{i=1}^{m} a_i \, y_i \, x_i, \qquad a_i \geq 0$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

So the new coordinates are never computed

- Put that w into the boundary:
  $w \cdot x + b = \Sigma\, a_i\, y_i\, (x_i \cdot x) + b$, so the pumps enter **only through inner products**
- A function giving the inner product **in the new coordinates**, from the original ones, is enough: a **kernel**
- The new coordinates are never computed: the RBF kernel, scikit-learn's default, costs one exponential per pair

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The kernel trick by hand: 25 = 25

Route, for a = (1, 2) and b = (3, 1)	Computation	Result
Lift with $\varphi(x) = (x_1^2, \sqrt{2}\cdot x_1 x_2, x_2^2)$, then multiply	$\varphi(a) \cdot \varphi(b) = (1, 2.83, 4) \cdot (9, 4.24, 1) = 9 + 12 + 4$	**25**
Never lift: the kernel $(a \cdot b)^2$	$(1\times3 + 2\times1)^2 = 5^2$	**25**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The RBF kernel: similarity falls with distance

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right)$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Training: one weight per pump

- RBF SVM on the standardised pumps, defaults $\gamma = 0.5$ and $C = 1$: one weight a_i per pump
- **922** pumps get weight 0: outside the slab, they drop out
- **268** get the maximum, $C = 1$ (inside the slab or on the wrong side); **10** sit on its edge. Together: 278 support vectors
- The model kept: those 278 pumps, their weights, and $b = +1.105$

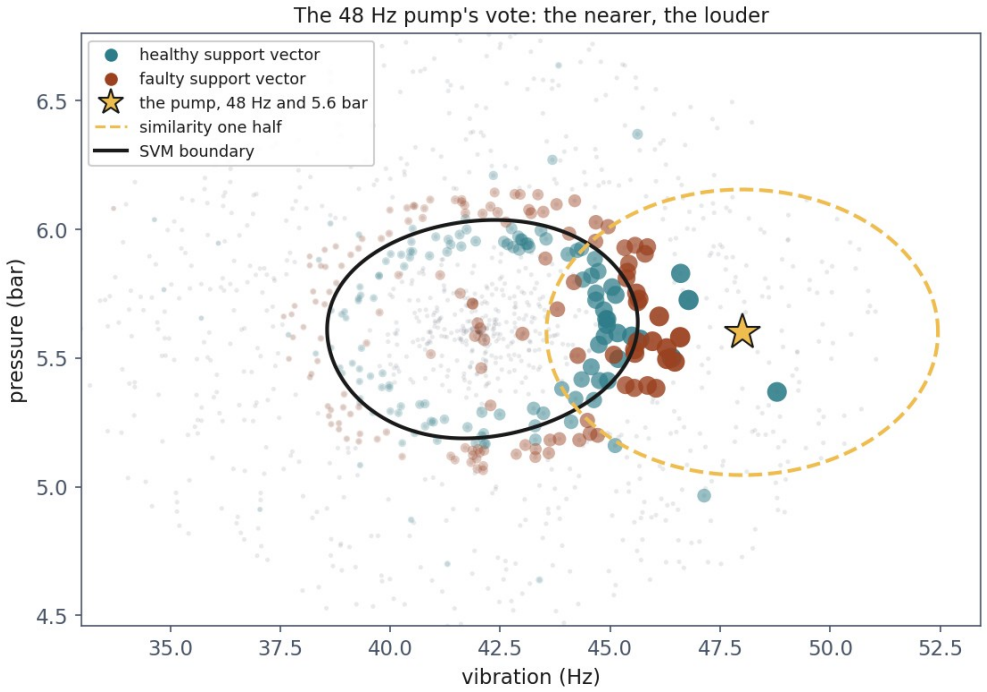
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The SVM on 48 Hz, 5.6 bar: +1.883

Term, for the pump x at 48 Hz, 5.6 bar	Value
Faulty support vectors: sum of $a_i \cdot K(x_i, x)$	+37.455
Healthy support vectors: minus the sum of $a_i \cdot K(x_i, x)$	−36.677
Intercept b	+1.105
Total: positive, so faulty	**+1.883**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

One pump’s vote, drawn

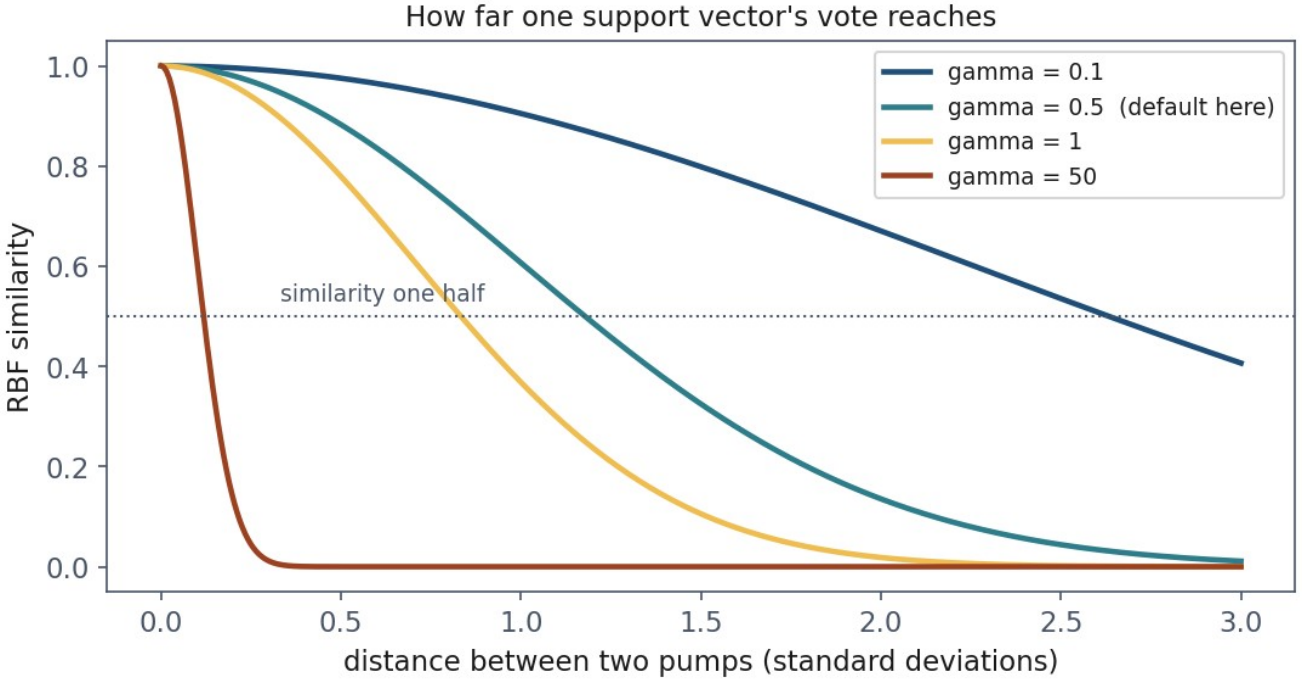


UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The highest training score, the lowest cross-validated one

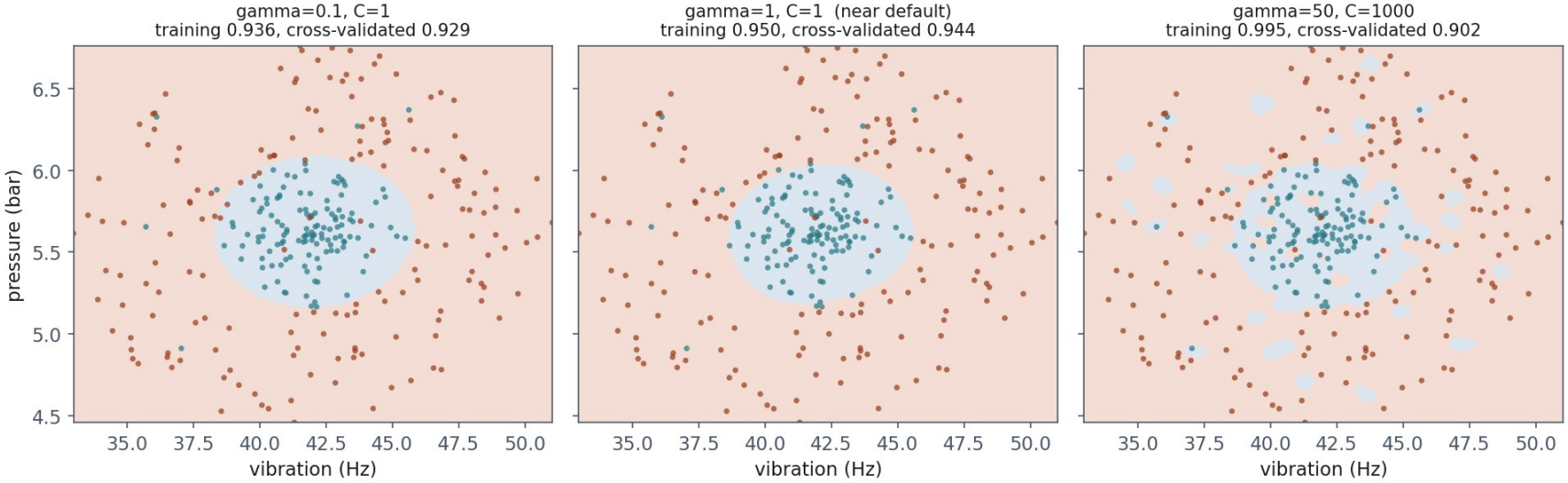
RBF SVM on the pumps	Training accuracy	Cross-validated accuracy, 5 folds
$\gamma = 0.1, C = 1$	0.936	0.929
$\gamma = 1, C = 1$	0.950	**0.944**
$\gamma = 50, C = 1000$	**0.995**	0.902

How far one support vector’s vote reaches, for four values of γ



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Overfitting you can see: the boundary at the three settings



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Notebook 3, live

section, and its slides	you call (→ the number to come back with)
the widest slab, 45-48	`SVC(kernel="linear")`, `.n_support_`, `decision_function` → 3 of 80 points
a straight line still fails, 51-52	`cross_val_score`, linear against RBF kernel → 947 against 278 support vectors
the kernel trick, 53-58	NumPy only: $(a \cdot b)^2$ against the lifted product → 25 = 25
training and one prediction, 59-61	`.support_`, `.dual_coef_`, `.intercept_` → +1.883
gamma and C, 62-64	`SVC(gamma=..., C=...)` in the pipeline → 0.944 at $\gamma = 1$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The three families compared, on the same pumps and the same folds

Model	Accuracy, 5-fold cross-validation
Majority baseline	0.613
Logistic regression (lesson 4)	0.613
SVM, linear kernel	0.613
Gaussian Naive Bayes	0.933
k-NN, k = 5	0.944
SVM, RBF kernel	**0.947**
noise ceiling	*≈ 0.96*

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

How to choose, in practice

Situation	Reach for
Few features, plenty of data, a curved boundary	k-NN, or an RBF SVM
Very many features, little data	Naive Bayes
You need probabilities you can trust	**Not** Naive Bayes
The model must be small, or fast at prediction	SVM, not k-NN
The signal is an interaction between features	k-NN or an RBF SVM, not Naive Bayes

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What to take away

- **No line separates the pumps**: 0.613, the majority share
- **k trades bias for variance**
- **In 100 dimensions the nearest point is 70% as far as the farthest**
- **Naive Bayes**: 0.933 on the pumps, **0.404** with interacting sensors
- **Choosing the family beats tuning**: 0.613 to 0.947

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Homework

- **Exercise 6**, discussed at the start of **Friday 20 November**
- You get a dataset, fit the three families, and argue which one you would choose, and why